Lawyer Contract API

Complete Purpose, Architecture, API Process, and Workflow Diagram

---

Technology: FastAPI, MongoDB, PyPDF2, Pydantic, Google Gemini, and the Python google-genai SDK

Application Purpose

The Lawyer Contract API converts a legal contract uploaded as PDF or TXT into searchable text and structured AI analysis. It provides a backend service for contract management and preliminary contract review.

The application does not replace professional legal advice. It organizes contract information and highlights possible areas that require human legal review.

What the User Can Do

Step	Capability	Result
1	Upload a PDF or TXT contract	Contract is saved and parsed
2	List uploaded contracts	Metadata list is returned
3	Retrieve one contract	Contract details are returned by ID
4	Request AI analysis	Gemini analyzes the extracted text
5	Receive structured insight	Summary, clauses, risks, and recommendations are returned
6	Store the result	Analysis is retained in MongoDB

High-Level Architecture

```
Client / Swagger UI
|
v
FastAPI Application
| |
v v
Contract Analysis
Routes Routes
| |
v v
Document Google Gemini
Parser google-genai SDK
| |
+------ MongoDB -----+
contracts | analysis
```

Core Components

Component	Purpose
app/main.py	Creates FastAPI application, initializes startup services, and registers routers.
app/routes/contracts.py	Handles upload, contract listing, and contract retrieval.
app/routes/analysis.py	Starts AI analysis, tracks status, and stores generated results.
app/service/document_parser.py	Extracts text and metadata from PDF and TXT files.
app/service/gemini_analyze.py	Builds the Google Gemini SDK request and validates response JSON.
app/service/prompt.py	Defines the legal-analysis instruction sent to Gemini.
app/models.py	Defines contract, clause, risk, and analysis Pydantic models.
app/database.py	Connects to MongoDB and exposes contracts and analysis collections.
app/config.py	Loads environment configuration and application limits.
docker-compose.yml	Starts the MongoDB service and persistent database volume.

Startup Workflow

```
1. Docker starts MongoDB on port 27017.
2. Uvicorn starts the FastAPI application.
3. app/config.py loads the .env configuration.
4. MongoDB connection is created for database 'mydb'.
5. Collections 'contracts' and 'analysis' are selected.
6. Unique indexes are created for contract_id and analysis_id.
7. Contract and analysis routers are registered.
8. The API becomes available through HTTP and Swagger UI.
```

API Routes

Method and Route	Action
GET /	Return application information and root endpoint summary.
POST /contracts/upload	Upload, validate, save, parse, and store a PDF or TXT contract.
GET /contracts/	List uploaded contracts without returning full extracted text.
GET /contracts/{contract_id}	Retrieve one contract using its generated contract_id or MongoDB _id.
POST /analysis/analyze/{contract_id}	Analyze a contract using Google Gemini.
POST /analysis/analyse/{contract_id}	British-spelling alias for the same analysis operation.

Document Upload Workflow

```
HTTP POST /contracts/upload
|
v
[Read multipart-uploaded file]
|
v
[Validate file extension: PDF or TXT]
|
valid / invalid
| |
v v
[Validate size] HTTP 400
<= 10 MB
|
v
[Generate UUID filename and contract_id]
|
v
[Save file in uploads/]
|
v
[Extract document text]
|
PDF TXT
| |
+-------+
|
v
[Calculate page count and word count]
|
v
[Insert contract record into MongoDB]
|
v
[Return contract_id and metadata]
```

Document Parsing Details

Format	Process	Output
TXT	Read the file as UTF-8 text	Text, one-page count, word count
PDF	Create PdfReader and extract text from each page	Combined text, actual page count, word count

MongoDB Data Model

contracts Collection

Field	Meaning
_id	MongoDB ObjectId, used internally.
contract_id	Public UUID-style application identifier.
filename	Unique stored filename in the uploads directory.
original_name	Original name provided by the uploader.
upload_date	ISO date and time of upload.
text_content	Complete extracted contract text.
page_count	Number of document pages or 1 for TXT.
word_count	Number of extracted text words.
status	Upload status; normally starts as uploaded.
analysis_status	in_progress, completed, or failed during analysis.

analysis Collection

Field	Meaning
_id	MongoDB ObjectId for the stored analysis.
analysis_id	Unique generated UUID-style analysis identifier.
contract_id	Contract ID associated with the analysis.
analysis_date	ISO date and time of AI analysis.
summary	Two-to-three sentence explanation.
contract_type	Example: NDA, Employment Contract, Service Agreement.
key_clauses	Important clauses, text, explanations, and standard-status flags.
risk_flags	Potential risks with descriptions and recommendations.
overall_risk_level	low, medium, high, or critical.
recommendations	List of recommended next actions.

AI Analysis Workflow

```
HTTP POST /analysis/analyze/{contract_id}
|
v
[Check Gemini API-key configuration]
|
v
[Find contract by contract_id or MongoDB _id]
|
v
[Verify extracted text exists]
|
v
[Set analysis_status = in_progress]
|
v
[Select first 15,000 contract characters]
|
v
[Send structured legal-analysis prompt to Gemini]
|
v
[google-genai async generate_content call]
|
v
[Receive JSON analysis]
|
v
[Remove code fences if present]
|
v
[Validate with Pydantic models]
|
success / failure
| |
v v
[Store result] [Set status = failed]
[Status = completed] HTTP 502
```

Gemini Response Structure

The prompt requests summary, contract type, key clauses, risk flags, overall risk level, and recommendations. Clause records contain a title, clause text, explanation, and standard-clause indicator. Risk records contain a title, description, risk level, recommendation, and clause reference.

Complete End-to-End Process

```
1. Client uploads a PDF or TXT contract.
2. FastAPI validates its extension and maximum file size.
3. The file is saved under uploads/.
4. PyPDF2 or the TXT reader extracts text.
5. Contract metadata, text, page count, and word count are stored.
6. The application returns a generated contract_id.
7. User retrieves the contract through a list or ID endpoint.
8. User requests AI analysis by contract_id.
9. The application updates the contract status to in_progress.
10. The service sends a bounded text prompt to Gemini.
11. Gemini returns a structured legal-analysis object.
12. Pydantic validates clauses and risk levels.
13. A unique analysis_id and MongoDB record are created.
14. The contract status becomes completed or failed.
15. The API returns the structured analysis result.
```

Analysis Status Lifecycle

Status	Meaning
uploaded	Document was stored and parsed, but analysis has not been requested.
in_progress	A Gemini analysis request is currently running.
completed	Gemini analysis succeeded and its result was stored.
failed	Gemini, parsing, validation, or another analysis operation failed.

Error Handling

Invalid extensions and oversized files return HTTP 400 responses. Missing contracts return HTTP 404. A missing Gemini key returns HTTP 500. Analysis provider failures are reported as HTTP 502, and the contract status is changed to failed so later retrieval can expose the failure state.

Example Process Sequence

```
POST /contracts/upload
-> contract_id: abc123

POST /analysis/analyze/abc123
-> status: in_progress
-> Gemini analysis
-> status: completed

GET /contracts/abc123
-> contract metadata and extracted text

Stored in MongoDB:
contracts record + analysis record
```